

AgriLink — API Integration (Week 5)

Flutter mobile application · Focus: `OpenStreetMap` routing and geocoding for pooled farm deliveries

1. API Name

`OpenStreetMap (OSM)` and its open service ecosystem — three services, none needing an API key or account: `OSRM` (`router.project-osrm.org`) for driving routes, distance, and ETA; `Nominatim` (`nominatim.openstreetmap.org`) for reverse geocoding; and the `OSM tile server` for the map imagery. OSM was chosen because AgriLink is a **logistics** platform — its core problem is moving bulk produce from farms to buyers, which is a routing problem. Google Maps requires a billing-enabled key; the OSM stack is free, open-licensed, and needs no secret in the repository. A fourth API, `Open-Meteo`, supplies weather (§6).

2. API Endpoints Used

```
GET https://router.project-osrm.org/route/v1/driving/{lon,lat};{lon,lat};...?overview=full&geometries=geojson
GET https://nominatim.openstreetmap.org/reverse?format=jsonv2&lat=15.4864&lon=120.9673&zoom=18
GET https://tile.openstreetmap.org/{z}/{x}/{y}.png
```

OSRM takes coordinates as `longitude,latitude` — the reverse of most APIs — joined with `;`, one waypoint per stop. `overview=full` returns the road geometry so the route can be drawn, not merely measured.

3. Sample JSON Response (actual, abbreviated)

```
{ "code": "Ok",
  "routes": [{ "distance": 49426, "duration": 2893.8,
    "geometry": { "coordinates": [[120.941396, 15.079411], [120.967269, 15.486488]] } },
  "waypoints": [ { "location": [120.941396, 15.079411], "name": "" },
    { "location": [120.967269, 15.486488], "name": "General Tinio Street" } ] }
```

`distance` is in metres and `duration` in seconds; both are converted before display. `geometry.coordinates` is a `[lon, lat]` list flipped to `[lat, lng]` for the map.

4. Explanation of the Integration

```
Farm + buyer coordinates (mobileOrders, Firestore) → RouteService.route(stops) --HTTP GET--> router.project-osrm.org
→ jsonDecode() → RoutePath { points, distanceKm, duration } → polyline on OSM tiles + "52 km • 52 min" on pool cards
```

How it improves the project. Before this, AgriLink measured a delivery as a **straight line** between farm and buyer. On provincial roads that understates the trip badly — the route in screenshot 1 is **52 km of road against roughly 45 km of straight line**, and that gap is the rider's unpaid fuel and time. The routing API supplies three things the app cannot compute alone:

- 1. A **distance a rider can trust** before accepting a pooled route.
- 2. An **ETA** (52 min), so a buyer can be told when produce arrives.
- 3. A **drawn route**, so the rider sees real roads instead of a line through rice fields.

HTTP GET: `client.get(uri, headers: {'User-Agent': ...}).timeout(Duration(seconds: 15))` — both OSRM and Nominatim require the client to identify itself. **JSON processing:** the payload never reaches the UI raw; it is decoded into a typed `RoutePath`, converting units and flipping every coordinate pair. **Displayed in:** the Pooled route map, the Order Pool cards, and the signup location picker.

5. Error Handling

Requirement	Failure	Detection	Result
Connection issue	No network / host unreachable	<code>SocketException</code>	All degrade to the same straight-line estimate at an assumed 28 km/h, flagged <code>isEstimate</code> . Failed routes are never cached, so the next refresh retries.
	Connection dropped mid-request	<code>ClientException</code>	
Unavailable API	Server down	HTTP 5xx	
	Rate limited	HTTP 429	
	Server too slow	<code>TimeoutException</code> (15 s)	
	No road between points	<code>empty routes</code>	
Invalid response	HTML error page where JSON expected	<code>FormatException</code>	
	Valid JSON, unexpected shape	cast failure	

Each failure is caught **separately and named** through a `RouteFailure` enum. The design rule is that a **failed lookup degrades to a usable number rather than an error** — a rider must never be blocked from seeing a distance. Nominatim failures fall back to the raw `lat, lng` so the pin stays usable, and with no connection at app start the splash screen offers a retry instead of hanging (screenshot 4).

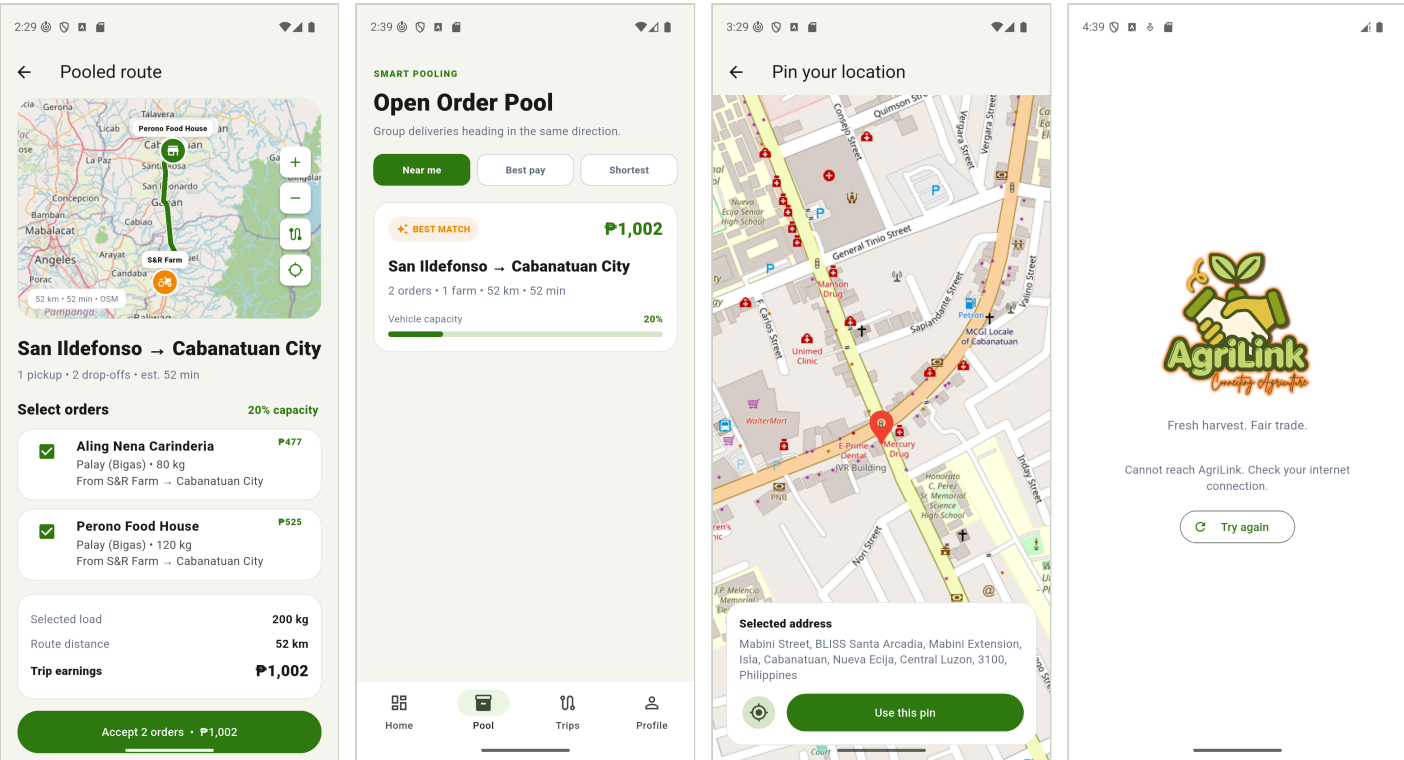
6. Bonus Features and Secondary Integration

Bonus	Implementation
Multiple endpoints	Five: OSRM route, Nominatim reverse, OSM tiles, Open-Meteo forecast, Open-Meteo geocoding
Search / filter	Town search over Open-Meteo geocoding, debounced 450 ms; Order Pool filters by Near me / Best pay / Shortest
POST request	OSRM, Nominatim, and Open-Meteo are read-only and expose no POST. The app does POST to the Firestore REST API (<code>documents:runQuery</code> , document creation) and to the transactional email API used by password reset.

Dynamic refresh	Pool routes resolve after paint and update in place via <code>setState</code> ; weather refreshes on a 10-minute timer, a refresh button, and pull-to-refresh — no page reload
-----------------	--

Open-Meteo (secondary): current conditions and a 7-day outlook for the coordinates an account pinned at signup, shown on the Farmer and Rider dashboards. The numeric WMO weather code is mapped to role-specific advice — the same thunderstorm tells a farmer to secure stored produce and a rider not to accept trips. **Testing:** `flutter test` runs **64 tests** against a mocked HTTP client; twelve cover OSRM, including every row of the error table. `flutter analyze` reports no issues. No new database collection was needed — routing reads coordinates already stored on `mobileOrders`.

7. Screenshots



1. OSRM route on OSM tiles — "52 km • 52 min • OSM"
2. Order Pool — real driving distance and ETA
3. Nominatim reverse geocoding in the picker
4. Error handling — offline, with retry

8. Reflection

Integrating the OpenStreetMap stack turned AgriLink's pooled delivery feature from an estimate into something a rider can actually trust: the Open Order Pool used to measure a route as a straight line, which on provincial roads understates the real trip by several kilometres of unpaid fuel and time. The hardest part was that OSRM returns coordinates as `[longitude, latitude]` — the reverse of almost every other API and of the mapping library we draw with — so every point must be flipped between the two systems, and a single missed flip puts the whole route in the wrong hemisphere. A second challenge was being a responsible consumer of a free public service: our pool screen reloads every few seconds, and the obvious implementation would have fired a burst of routing requests on every refresh, so routes resolve one at a time, only for what is on screen, and are cached by waypoint list. The most valuable lesson was that an integration is judged by how it fails, not how it succeeds — the routing service is allowed to be down, and when it is, the app quietly falls back to a straight-line estimate rather than showing a rider an error where a distance should be.